

Task – 6 TCP SYN Scanner

Development and Execution Observations

Name – Atish Kadam

Roll No - CS25MTECH14003

1. Project Overview

The TCP SYN Scanner is a Python-based network reconnaissance tool built using Scapy. It performs stealth port scanning by sending TCP SYN packets and analyzing responses to determine port states without completing the full TCP handshake.

1.1 Key Features

- Stealth scanning using SYN packets (half-open connections)
- Multi-threaded architecture for concurrent port scanning
- Configurable timeout and thread pool parameters
- Flexible port specification (ranges, lists, or mixed)
- Real-time output with live port status updates
- Common service identification for discovered open ports

2. Development Observations

2.1 Architecture Decisions

Thread-Based Concurrency: The scanner uses Python threading with semaphore-based limiting rather than `async/await`. This design choice provides:

- Better compatibility with Scapy's synchronous API
- Simpler mental model for concurrent network operations
- Fine-grained control over resource consumption via semaphore limits

2.2 Performance Considerations

Timeout Tuning: The default 1-second timeout balances responsiveness with accuracy. Shorter timeouts risk false filtered results; longer timeouts slow down large scans unnecessarily.

Thread Pool Sizing: Default 100 threads provides good throughput on most networks. Higher thread counts (200+) can saturate network capacity or trigger IDS/IPS systems.

3. Execution Observations and Test Results

3.1 Know the router

```
(atishkadam158@AtishKadam158) - [~/Desktop/All NS/Class participatory/Task-6]
• $ ip route | grep default
default via 192.168.0.1 dev wlan0 proto dhcp src 192.168.0.106 metric 600
```

3.2 Test Case 1: Default Port Range (1-1024)

Command: `sudo python3 tcp_syn_scanner.py 192.168.0.1`

```
(atishkadam158@AtishKadam158) - [~/Desktop/All NS/Class participatory/Task-6]
• $ sudo python3 tcp_syn_scanner.py 192.168.0.1
[sudo] password for atishkadam158:
=====
TCP SYN Scanner
=====
Target   : 192.168.0.1
Ports    : 1024 ports
Threads  : 100
Timeout  : 1.0s per port
Started  : 2026-04-10 18:25:21
=====

[OPEN]    Port    53/tcp
[OPEN]    Port    80/tcp
[OPEN]    Port    22/tcp
=====

SCAN SUMMARY
=====
Scan completed in : 14.59 seconds
Total ports scanned: 1024

OPEN      (3 ports)
  22/tcp  → SSH
  53/tcp  → DNS
  80/tcp  → HTTP

FILTERED (1021 ports) – no response
=====
```

3.3 Test Case 2: Increased Timeout (2 seconds)

Command: `sudo python3 tcp_syn_scanner.py 192.168.0.1 -p 1-1024 --timeout 2`

```
(atishkadam158@AtishKadam158)-[~/Desktop/All NS/Class participatory/Task-6]
$ sudo python3 tcp_syn_scanner.py 192.168.0.1 -p 1-1024 --timeout 2

=====
TCP SYN Scanner
=====
Target   : 192.168.0.1
Ports    : 1024 ports
Threads  : 100
Timeout  : 2.0s per port
Started  : 2026-04-10 18:27:29
=====

[OPEN]    Port    22/tcp
[OPEN]    Port    80/tcp
=====

SCAN SUMMARY
=====
Scan completed in : 26.52 seconds
Total ports scanned: 1024

OPEN      (2 ports)
  22/tcp  → SSH
  80/tcp  → HTTP

FILTERED (1022 ports) – no response
=====
```

3.4 Test Case 3: Specific Port List

Command: `sudo python3 tcp_syn_scanner.py 192.168.0.1 -p 22,80,443,3306`

```
(atishkadam158@AtishKadam158)-[~/Desktop/All NS/Class participatory/Task-6]
$ sudo python3 tcp_syn_scanner.py 192.168.0.1 -p 22,80,443,3306

=====
TCP SYN Scanner
=====
Target   : 192.168.0.1
Ports    : 4 ports
Threads  : 100
Timeout  : 1.0s per port
Started  : 2026-04-10 18:25:57
=====

[OPEN]    Port    22/tcp
[OPEN]    Port    80/tcp
=====

SCAN SUMMARY
=====
Scan completed in : 1.14 seconds
Total ports scanned: 4

OPEN      (2 ports)
  22/tcp  → SSH
  80/tcp  → HTTP

FILTERED (2 ports) – no response
=====
```

3.5 Test Case 4: Small Port Range (1-55)

Command: `sudo python3 tcp_syn_scanner.py 192.168.0.1 --ports 1-55`

```
(atishkadam158@AtishKadam158) - [~/Desktop/All NS/Class participatory/Task-6]
$ sudo python3 tcp_syn_scanner.py 192.168.0.1 --ports 1-55
=====
TCP SYN Scanner
=====
Target   : 192.168.0.1
Ports    : 55 ports
Threads  : 100
Timeout  : 1.0s per port
Started  : 2026-04-10 18:25:47
=====

[OPEN]    Port    53/tcp
[OPEN]    Port    22/tcp

=====

SCAN SUMMARY
=====
Scan completed in : 2.06 seconds
Total ports scanned: 55

OPEN      (2 ports)
  22/tcp  → SSH
  53/tcp  → DNS

FILTERED (53 ports) – no response
=====
```

3.6 Test Case 5: Extended Range with High Thread Count

Command: `sudo python3 tcp_syn_scanner.py 192.168.0.1 -p 1-6553 -t 200`

```
(atishkadam158@AtishKadam158) - [~/Desktop/All NS/Class participatory/Task-6]
$ sudo python3 tcp_syn_scanner.py 192.168.0.1 -p 1-6553 -t 200
=====
TCP SYN Scanner
=====
Target   : 192.168.0.1
Ports    : 6553 ports
Threads  : 200
Timeout  : 1.0s per port
Started  : 2026-04-10 18:26:18
=====

[OPEN]    Port    53/tcp
[OPEN]    Port    22/tcp
[OPEN]    Port    80/tcp
[OPEN]    Port   1900/tcp

=====

SCAN SUMMARY
=====
Scan completed in : 45.44 seconds
Total ports scanned: 6553

OPEN      (4 ports)
  22/tcp  → SSH
  53/tcp  → DNS
  80/tcp  → HTTP
  1900/tcp → unknown

FILTERED (6549 ports) – no response
=====
```

4. Performance Analysis

4.1 Thread Scaling Efficiency

Thread count directly impacts scan speed. The test results show near-linear scaling:

- **100 threads:** ~70 ports/second
- **200 threads:** ~144 ports/second (2.06x improvement)

4.2 Timeout Impact

Timeout settings significantly affect scan duration. For local network scans where response times are predictable, the default 1-second timeout is optimal. Increasing to 2 seconds nearly doubled scan time without discovering additional ports, indicating diminishing returns for longer timeouts on responsive networks.

4.3 Port Discovery Consistency

Across multiple test runs, the following ports were consistently discovered:

- **Port 22 (SSH):** Detected in all scans
- **Port 53 (DNS):** Detected in most scans (some timing variance observed)
- **Port 80 (HTTP):** Consistently detected when scanned
- **Port 1900 (UPnP):** Only discovered in extended range scan

5. Conclusions

The TCP SYN scanner demonstrates effective and stealthy port reconnaissance with accurate service detection, leveraging a multi-threaded design to scan thousands of ports quickly, especially on local networks. Optimal configurations vary by scenario, with higher thread counts and shorter timeouts suited for local scans, moderate settings for remote networks, and lower thread counts for stealth operations to reduce detection risk. Future enhancements could include service version detection, adaptive timeouts, multiple export formats, OS fingerprinting, and progress indicators for long scans. However, it is crucial to use this tool responsibly and only for authorized security assessments, as unauthorized scanning may violate legal and organizational policies.

Task – 10 ARP Spoof Detector

Development & Execution Observations

1. Project Overview

arp_spoof_detector.py is a Python 3 tool built with Scapy that passively monitors ARP traffic on a network interface and raises real-time alerts when it detects signs of ARP spoofing or cache poisoning. It logs events both to a structured .log file and a CSV file for post-analysis.

2. Environment

- OS: Linux (Arch-based, wlan0 interface)
- Tool: arp_spoof_detector.py — run with `sudo python3`
- Attacker simulation: `arpspoof -i wlan0 192.168.0.1`
- Gateway: 192.168.0.1 (MAC: 28:87:ba:d8:07:2c — auto-detected via ARP)
- Attacker MAC (spoofed): ac:12:03:06:03:e2

3. Development Observations

3.1 Architecture Decisions

On startup, the detector queries `ip route` to find the default gateway IP, then sends an ARP request to resolve its MAC. This pre-loads the ARP table so any impersonation attempt is caught immediately. Both values were successfully auto-detected (192.168.0.1 / 28:87:ba:d8:07:2c) without manual flags.

- **Real-Time Packet Sniffing**
- **Rule-Based Detection**
- **Modular Design**
- **Stateful Monitoring**

3.2 Performance Considerations

- **Selective Packet Filtering:** The tool processes only ARP packets, reducing unnecessary computation and improving efficiency during monitoring.
- **Lightweight Processing:** Each packet is checked using simple comparisons and threshold-based logic. This keeps runtime overhead low and makes the detector suitable for continuous operation.
- **Immediate Logging:** Alerts are written to both the log file and CSV file as soon as they are detected. This ensures that important events are preserved without delay and supports later analysis.

- **Threshold Tuning:** The gratuitous ARP flood detection uses a time window and count threshold. These values balance sensitivity and false positives, since very low thresholds may produce extra alerts, while very high thresholds may miss real attacks.
- **Graceful Shutdown:** Signal handling is used to close files properly and print a final summary when the program stops. This prevents data loss and keeps execution clean.

4. Execution Results

The attacker ran arpspoof broadcasting fake ARP replies claiming 192.168.0.1 is at ac:12:03:06:03:e2. The detector raised the following alert types:

Alert Type	Description
BROADCAST REPLY	ARP reply sent to broadcast address (ff:ff:ff:ff:ff:ff) — uncommon, likely spoofing indicator.
GATEWAY IMPERSONATION	A host claims to be the gateway (192.168.0.1) using a different MAC than the known legitimate one.
MAC CHANGE	An IP address previously mapped to one MAC is now advertising a different MAC — strong ARP spoofing indicator.
GRATUITOUS ARP FLOOD	A host sent 5+ gratuitous ARP replies within 10 seconds — indicative of ARP cache poisoning.
LAYER MISMATCH	Ethernet frame source MAC differs from ARP sender MAC — possible packet crafting / spoofing.

```
(atishkadam158@AtishKadam158) - [~/Desktop/All NS/Class participatory/Task-10]
$ sudo python3 arp_spoof_detector.py
[sudo] password for atishkadam158:
=====
      ARP SPOOF DETECTOR  v1.0
    Monitors ARP replies for inconsistencies
=====

Auto-detecting gateway ...
[+] Gateway IP   : 192.168.0.1
[+] Gateway MAC  : 28:87:ba:d8:07:2c

Logging events to : arp_detector.log
CSV alerts to     : arp_alerts.csv

Gateway pre-loaded: 192.168.0.1 -> 28:87:ba:d8:07:2c
Sniffing ARP on interface: wlan0
Press Ctrl+C to stop.
```

Result after Attack simulation starts-

```
(atishkadam158@AtishKadam158) - [~/Desktop/All NS/Class participatory/Task-10]
$ sudo python3 arp_spoof_detector.py
[sudo] password for atishkadam158:
=====
      ARP SPOOF DETECTOR  v1.0
    Monitors ARP replies for inconsistencies
=====

Auto-detecting gateway ...
[+] Gateway IP   : 192.168.0.1
[+] Gateway MAC  : 28:87:ba:d8:07:2c

Logging events to : arp_detector.log
CSV alerts to     : arp_alerts.csv

Gateway pre-loaded: 192.168.0.1 -> 28:87:ba:d8:07:2c
Sniffing ARP on interface: wlan0
Press Ctrl+C to stop.

=====
!! ALERT #1 [19:02:32] BROADCAST REPLY
=====
ARP reply sent to broadcast address -- uncommon, may indicate spoofing
Source IP   : 192.168.0.1
Claimed MAC : ac:12:03:06:03:e2

2026-04-10 19:02:32 | WARNING | [BROADCAST REPLY] ARP reply sent to broadcast address -- uncommon, may indicate spoofing | src=192.168.0.1 claimed=ac:12:03:06:03:e2 known=

=====
!! ALERT #2 [19:02:32] GATEWAY IMPERSONATION
=====
Someone is claiming to be the gateway (192.168.0.1)!
Source IP   : 192.168.0.1
Claimed MAC : ac:12:03:06:03:e2
Known MAC   : 28:87:ba:d8:07:2c

2026-04-10 19:02:32 | WARNING | [GATEWAY IMPERSONATION] Someone is claiming to be the gateway (192.168.0.1)! | src=192.168.0.1 claimed=ac:12:03:06:03:e2 known=28:87:ba:d8:07:2c

=====
!! ALERT #3 [19:02:32] MAC CHANGE
=====
IP 192.168.0.1 has changed its MAC address -- ARP spoofing suspected!
Source IP   : 192.168.0.1
Claimed MAC : ac:12:03:06:03:e2
Known MAC   : 28:87:ba:d8:07:2c

2026-04-10 19:02:32 | WARNING | [MAC CHANGE] IP 192.168.0.1 has changed its MAC address -- ARP spoofing suspected! | src=192.168.0.1 claimed=ac:12:03:06:03:e2 known=28:87:ba:d8:07:2c
```

Attacker Simulation

```
(atishkadam158@AtishKadam158) - [~/Desktop/All NS/Class participatory/Task-10]
$ sudo arpspoof -i wlan0 192.168.0.1
[sudo] password for atishkadam158:
ac:12:3:6:3:e2 ff:ff:ff:ff:ff:ff 0806 42: arp reply 192.168.0.1 is-at ac:12:3:6:3:e2
ac:12:3:6:3:e2 ff:ff:ff:ff:ff:ff 0806 42: arp reply 192.168.0.1 is-at ac:12:3:6:3:e2
```

5. Log & CSV Output

arp_detector.log captured timestamped WARNING entries for each alert and INFO entries for learned IPs. The CSV (arp_alerts.csv) recorded structured rows with columns: timestamp, alert_type, src_ip, claimed_mac, known_mac, interface — enabling easy filtering in Excel or pandas.

Logging at logger file -

```
arp_detector.log
1 2026-04-10 19:19:49,448 INFO | Detector started on wlan0 | gateway=192.168.0.1 mac=28:87:ba:d8:07:2c
2 2026-04-10 19:19:41,488 WARNING | [BROADCAST REPLY] ARP reply sent to broadcast address -- uncommon, may indicate spoofing | src=192.168.0.1 claimed=ac:12:03:06:03:e2 known=
3 2026-04-10 19:19:41,489 WARNING | [GATEWAY IMPERSONATION] Someone is claiming to be the gateway (192.168.0.1) | src=192.168.0.1 claimed=ac:12:03:06:03:e2 known=28:87:ba:d8:07:2c
4 2026-04-10 19:19:41,489 WARNING | [MAC CHANGE] IP 192.168.0.1 has changed its MAC address -- ARP spoofing suspected | src=192.168.0.1 claimed=ac:12:03:06:03:e2 known=28:87:ba:d8:07:2c
5 2026-04-10 19:19:43,485 WARNING | [BROADCAST REPLY] ARP reply sent to broadcast address -- uncommon, may indicate spoofing | src=192.168.0.1 claimed=ac:12:03:06:03:e2 known=
6 2026-04-10 19:19:43,486 WARNING | [GATEWAY IMPERSONATION] Someone is claiming to be the gateway (192.168.0.1) | src=192.168.0.1 claimed=ac:12:03:06:03:e2 known=28:87:ba:d8:07:2c
7 2026-04-10 19:19:43,771 INFO | Detector stopped by user
8 2026-04-10 19:21:08,068 INFO | Detector started on wlan0 | gateway=192.168.0.1 mac=28:87:ba:d8:07:2c
9 2026-04-10 19:21:14,600 WARNING | [BROADCAST REPLY] ARP reply sent to broadcast address -- uncommon, may indicate spoofing | src=192.168.0.1 claimed=ac:12:03:06:03:e2 known=
10 2026-04-10 19:21:14,600 WARNING | [GATEWAY IMPERSONATION] Someone is claiming to be the gateway (192.168.0.1) | src=192.168.0.1 claimed=ac:12:03:06:03:e2 known=28:87:ba:d8:07:2c
11 2026-04-10 19:21:14,600 WARNING | [MAC CHANGE] IP 192.168.0.1 has changed its MAC address -- ARP spoofing suspected | src=192.168.0.1 claimed=ac:12:03:06:03:e2 known=28:87:ba:d8:07:2c
12 2026-04-10 19:21:15,601 WARNING | [BROADCAST REPLY] ARP reply sent to broadcast address -- uncommon, may indicate spoofing | src=192.168.0.1 claimed=ac:12:03:06:03:e2 known=
13 2026-04-10 19:21:15,601 WARNING | [GATEWAY IMPERSONATION] Someone is claiming to be the gateway (192.168.0.1) | src=192.168.0.1 claimed=ac:12:03:06:03:e2 known=28:87:ba:d8:07:2c
14 2026-04-10 19:21:15,601 WARNING | [BROADCAST REPLY] ARP reply sent to broadcast address -- uncommon, may indicate spoofing | src=192.168.0.1 claimed=ac:12:03:06:03:e2 known=
15 2026-04-10 19:21:18,082 WARNING | [GATEWAY IMPERSONATION] Someone is claiming to be the gateway (192.168.0.1) | src=192.168.0.1 claimed=ac:12:03:06:03:e2 known=28:87:ba:d8:07:2c
16 2026-04-10 19:21:20,601 WARNING | [BROADCAST REPLY] ARP reply sent to broadcast address -- uncommon, may indicate spoofing | src=192.168.0.1 claimed=ac:12:03:06:03:e2 known=
17 2026-04-10 19:21:20,601 WARNING | [GATEWAY IMPERSONATION] Someone is claiming to be the gateway (192.168.0.1) | src=192.168.0.1 claimed=ac:12:03:06:03:e2 known=28:87:ba:d8:07:2c
18 2026-04-10 19:21:22,601 WARNING | [GRATUITOUS ARP FLOOD] 192.168.0.1 sent 5 gratuitous ARPs in 10s -- possible ARP cache poisoning | src=192.168.0.1 claimed=ac:12:03:06:03:e2 known=ac:12:03:06:03:e2
19 2026-04-10 19:21:22,602 WARNING | [BROADCAST REPLY] ARP reply sent to broadcast address -- uncommon, may indicate spoofing | src=192.168.0.1 claimed=ac:12:03:06:03:e2 known=
20 2026-04-10 19:21:22,602 WARNING | [GATEWAY IMPERSONATION] Someone is claiming to be the gateway (192.168.0.1) | src=192.168.0.1 claimed=ac:12:03:06:03:e2 known=28:87:ba:d8:07:2c
21 2026-04-10 19:21:24,602 WARNING | [BROADCAST REPLY] ARP reply sent to broadcast address -- uncommon, may indicate spoofing | src=192.168.0.1 claimed=ac:12:03:06:03:e2 known=
22 2026-04-10 19:21:24,603 WARNING | [GATEWAY IMPERSONATION] Someone is claiming to be the gateway (192.168.0.1) | src=192.168.0.1 claimed=ac:12:03:06:03:e2 known=28:87:ba:d8:07:2c
23 2026-04-10 19:21:32,819 INFO | Learned: 192.168.0.183 -> 76:62:93:ff:43:90
24
```

Logging at CSV File

```
timestamp,alert_type,src_ip,claimed_mac,known_mac,interface
2026-04-10 19:21:14,BROADCAST REPLY,192.168.0.1,ac:12:03:06:03:e2,,wlan0
2026-04-10 19:21:14,GATEWAY IMPERSONATION,192.168.0.1,ac:12:03:06:03:e2,28:87:ba:d8:07:2c,wlan0
2026-04-10 19:21:14,MAC CHANGE,192.168.0.1,ac:12:03:06:03:e2,28:87:ba:d8:07:2c,wlan0
2026-04-10 19:21:16,BROADCAST REPLY,192.168.0.1,ac:12:03:06:03:e2,,wlan0
2026-04-10 19:21:16,GATEWAY IMPERSONATION,192.168.0.1,ac:12:03:06:03:e2,28:87:ba:d8:07:2c,wlan0
2026-04-10 19:21:18,BROADCAST REPLY,192.168.0.1,ac:12:03:06:03:e2,,wlan0
2026-04-10 19:21:18,GATEWAY IMPERSONATION,192.168.0.1,ac:12:03:06:03:e2,28:87:ba:d8:07:2c,wlan0
2026-04-10 19:21:20,BROADCAST REPLY,192.168.0.1,ac:12:03:06:03:e2,,wlan0
2026-04-10 19:21:20,GATEWAY IMPERSONATION,192.168.0.1,ac:12:03:06:03:e2,28:87:ba:d8:07:2c,wlan0
2026-04-10 19:21:22,GRATUITOUS ARP FLOOD,192.168.0.1,ac:12:03:06:03:e2,ac:12:03:06:03:e2,wlan0
2026-04-10 19:21:22,BROADCAST REPLY,192.168.0.1,ac:12:03:06:03:e2,,wlan0
2026-04-10 19:21:22,GATEWAY IMPERSONATION,192.168.0.1,ac:12:03:06:03:e2,28:87:ba:d8:07:2c,wlan0
2026-04-10 19:21:24,BROADCAST REPLY,192.168.0.1,ac:12:03:06:03:e2,,wlan0
2026-04-10 19:21:24,GATEWAY IMPERSONATION,192.168.0.1,ac:12:03:06:03:e2,28:87:ba:d8:07:2c,wlan0
```

6. Conclusion

The detector successfully identified all forms of ARP spoofing activity introduced by the simulated attacker. Alerts fired within milliseconds of the first spoofed packet. The combination of log and CSV output provides both human-readable and machine-parseable records for incident response and forensic review.